КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики
Кафедра теорії та технології програмування

Звіт до навчального проєкту № 1
з дисципліни «Методи паралельних обчислень»
на тему «Паралельна побудова зворотного індексу»

Виконав студент 4-го курсу
Групи ТТП-42
Тимофій ЛЕЙЦИШИН

Київ — 2026

1. УМОВИ ПРОВЕДЕННЯ ЕКСПЕРИМЕНТУ

Конфігурацію обчислювальної системи та умови запуску наведено в табл. 1.

Таблиця 1 – Умови проведення експерименту

Параметр	**Значення**
Процесор	Apple M1 Max (ARM64): 10 ядер без SMT, з них 8 продуктивних (P, два кластери по 4 ядра) і 2 енергоефективні (E)
Кеш-пам’ять	L1 даних: 128 КБ на P-ядро, 64 КБ на E-ядро; L2: 12 МБ на P-кластер, 4 МБ на E-кластер; системний кеш (SLC) 48 МБ
Рядок кешу	128 байт (за даними `sysctl hw.cachelinesize`; підтверджено дослідженням хибного розділення)
Оперативна пам’ять	32 ГБ LPDDR5
ОС і компілятор	macOS 26.6.2 (25G83); Go 1.26.2 darwin/arm64
Збирання	`go build -trimpath`; оптимізація та вбудовування функцій увімкнені за замовчуванням (аналог `-O2`), без `-race` і `-gcflags=-N -l`
Умови запуску	20.10.2026, 00:24–00:44; живлення від мережі, режим енергозбереження вимкнено; сторонні застосунки закрито; середнє навантаження системи (load average) на початку вимірювань становило 3,4

1.1 Корпус текстів

Корпус містить 722 книги англійською мовою з Project Gutenberg, завантажені скриптом `scripts/corpus.sh` через інтерфейс масового завантаження (robot harvest). Обсяг корпусу становить 242 722 532 байти (231,5 МіБ), розмір файлів коливається від 30 КБ до 2,4 МБ (медіана 260 КБ). Усі файли перевірено на коректність UTF-8; файли в кодуванні Latin-1 не використовувалися. Після нормалізації корпус містить 32 284 949 токенів і 495 761 різне слово. Розподіл частот слів відповідає закону Ципфа: на 20 найчастотніших слів припадає 26,75 % усіх входжень, зокрема 7,61 % на слово `the`.

2. МЕТОДИКА ВИМІРЮВАНЬ

- Час побудови індексу вимірюється за монотонним годинником. Корпус зчитується в оперативну пам’ять до вимірювання, сортування і запис результату виконуються після нього. Час злиття у V4 входить до часу побудови і додатково вимірюється окремо.
- Кожну конфігурацію (версія, кількість потоків, K) вимірювано окремим запуском програми, яка зчитує корпус і будує індекс шість разів. Перша побудова до результатів не включається, наступні п’ять є вимірюваними повтореннями. Індекс, побудований у попередньому повторенні, не зберігається, а перед кожним повторенням виконується збирання сміття, тому всі повторення починаються з однаковим обсягом зайнятої пам’яті.
- Для кожної конфігурації наведено медіану п’яти повторень, а також мінімальне і максимальне значення як характеристику розкиду. Усі результати записано у файл `results.csv` (стовпці `program, version, threads, k, run, seconds`). Прискорення та ефективність обчислюються відносно V0: S(P) = T(V0) / T(P), E(P) = S(P) / P.
- Вихідний файл кожного запуску побайтово порівнюється з вихідним файлом V0; запусків із розбіжністю не було.

– Усі вимірювання виконує скрипт `scripts/final.sh`. Кількість потоків P ∈ {1, 2, 4, 6, 8, 10}; для V3 використано K = 1024, залежність від K досліджено за P = 8.

3. ОПИС РЕАЛІЗАЦІЇ

Програму написано мовою Go без сторонніх бібліотек. Спільний код міститься в пакеті `internal/index`; програма `cmd/invindex` реалізує всі версії побудови індексу, а програма `cmd/falseshare` призначена для дослідження хибного розділення. Паралельні потоки виконання реалізовано горутинами; середовище виконання розподіляє їх між потоками ОС, кількість яких дорівнює кількості ядер (GOMAXPROCS = 10).

3.1 Нормалізація тексту

Еталонну функцію нормалізації `Words` реалізовано засобами стандартної бібліотеки: `bytes.FieldsFuncSeq` виділяє максимальні послідовності літер і цифр (`unicode.IsLetter`, `unicode.IsDigit`); будь-який інший символ, зокрема некоректна послідовність UTF-8, вважається роздільником. Токени, коротші за три символи, відкидаються, решта переводиться в нижній регістр. Під час вимірювань використовується оптимізована функція `FastWords` (підрозд. 5.8); збіг її результатів з результатами `Words` перевіряють модульні тести і фазинг-тестування.

3.2 Структура індексу

Індекс реалізовано як хеш-таблицю `map[string]*Entry`. Структура `Entry` містить загальну кількість входжень слова `Total` і таблицю `Docs`, що зіставляє номеру документа кількість входжень у ньому. Одна операція оновлення відповідає одному входженню слова. Рядки вихідного файлу впорядковано за словом; записи в межах рядка впорядковано за спаданням кількості входжень, а за однакової кількості — за зростанням номера документа. Документи нумеруються з нуля в лексикографічному порядку шляхів до файлів.

3.3 Версії програми

Роботу між потоками розподілено за файлами: кожна з P горутин отримує номер наступного необробленого файлу зі спільного атомарного лічильника, що вирівнює навантаження за різного розміру файлів. Атомарна операція виконується один раз на файл і не стосується індексу. Реалізацію версій наведено в табл. 2.

Таблиця 2 – Реалізація версій програми

Версія	**Реалізація**
V0	один потік, один індекс
V1	спільна хеш-таблиця з методом ланцюжків без засобів синхронізації; стандартна `map` непридатна, бо за одночасного запису середовище Go аварійно завершує програму. Для кожного слова таблиця містить вузол із загальною кількістю входжень і по одному вузлу на кожен документ. Після створення вузла потоки змінюють лише поля розміром у машинне слово, тому гонка призводить до втрати оновлень, але не пошкоджує пам’ять
V2	один індекс і одна блокувальна змінна (`sync.Mutex`), яка захоплюється на час кожної операції оновлення

Версія	Реалізація
V3	K секцій, кожна з власною блокувальною змінною і власною хеш-таблицею; слово належить секції з номером hash(слово) mod K. Розмір секції доповнено заповнювачем до 128 байт, щоб блокувальні змінні різних секцій не потрапляли в один рядок кешу
V4	кожна горутина будує локальний індекс без синхронізації, після чого локальні індекси зливаються: у `v4seq` вони послідовно приєднуються до першого індексу (P − 1 крок), у `v4tree` зливаються попарно за схемою збалансованого дерева, причому злиття одного рівня виконуються паралельно ($\lceil \log_2 P \rceil$ кроків)
`cas` (додаткове завдання)	хеш-таблиця V1, у якій обидві операції, що спричиняли гонку, виконуються атомарно: вузол додається операцією порівняння з обміном (`CompareAndSwap`) з повтором у разі невдачі, лічильники збільшуються атомарно (`atomic.Int64.Add`)
`syncmap` (додаткове завдання)	структура даних `sync.Map` зі стандартної бібліотеки (реалізована як конкурентне префіксне дерево хешів) з ключем «слово, документ» і атомарним лічильником як значенням

3.4 Програма дослідження хибного розділення

P горутин виконують по n інкрементів власного лічильника типу `int64`. Лічильники розміщено в одному масиві із заданою відстанню між сусідніми: відстань 8 байт відповідає реалізації A (`long counters[P]`), відстань 64 байти відповідає реалізації B (`struct { long value; char pad[56]; }`); додатково досліджено відстані від 16 до 256 байт. Початок масиву вирівняно на межу 128 байт. Дизасемблювання підтверджує, що кожна ітерація циклу виконує читання з пам’яті і запис у пам’ять.

4. РЕЗУЛЬТАТИ

4.1 Масштабування

Час побудови індексу і прискорення наведено в табл. 3 і на рис. 1. Час V0 становить 2,992 с (від 2,933 до 3,050 с). Результати всіх повторень наведено у файлі `results.csv`.

Таблиця 3 – Медіана часу побудови індексу, с, і прискорення (у дужках)

P	V2	V3 (K = 1024)	V4, послідовне	V4, дерево	`cas`	`syncmap`
1	3,104 (0,96)	3,465 (0,86)	3,031 (0,99)	3,033 (0,99)	4,088 (0,73)	8,892 (0,34)
2	6,301 (0,47)	1,895 (1,58)	1,837 (1,63)	1,802 (1,66)	2,781 (1,08)	5,708 (0,52)
4	7,730 (0,39)	1,168 (2,56)	1,231 (2,43)	1,198 (2,50)	2,138 (1,40)	4,135 (0,72)
6	9,197 (0,33)	1,559 (1,92)	0,995 (3,01)	0,946 (3,16)	1,997 (1,50)	3,657 (0,82)
8	10,516 (0,28)	1,540 (1,94)	0,922 (3,25)	0,836 (3,58)	1,904 (1,57)	3,733 (0,80)
10	11,545 (0,26)	1,624 (1,84)	0,942 (3,18)	0,842 (3,55)	1,909 (1,57)	3,540 (0,85)

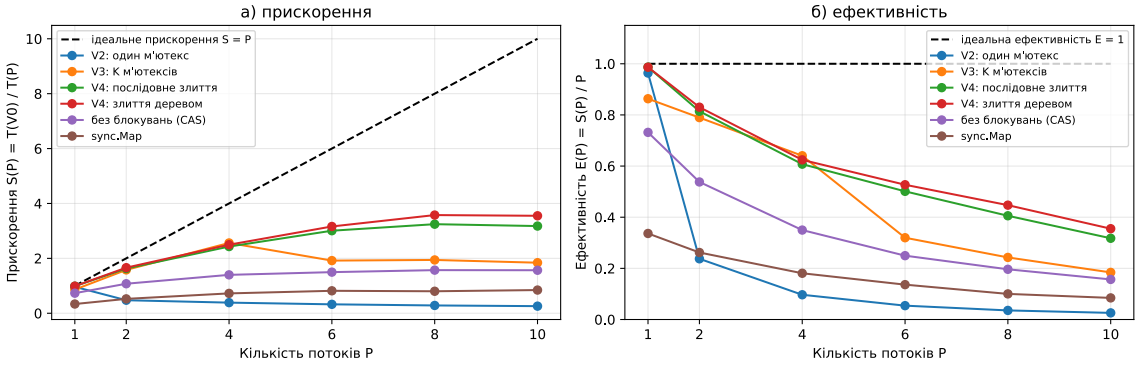


Рисунок 1 – Прискорення (а) та ефективність (б) залежно від кількості потоків

4.2 Втрата оновлень у V1

Результати десяти запусків V1 за P = 8 наведено в табл. 4; еталонна кількість входжень, отримана V0, становить 32 284 949. Жоден запуск не дав правильного результату, і результати всіх запусків різні. Середня частка втрачених оновлень зростає з кількістю потоків: 0,12 % за P = 2, 0,52 % за P = 4, 2,33 % за P = 8 і 2,74 % за P = 10 (по 10 запусків для кожного P).

Таблиця 4 – Втрачені оновлення у V1 за P = 8

Запуск	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Втрачено, тис.	785,2	746,8	800,5	802,5	727,6	808,2	717,9	597,0	749,9	789,3
Втрачено, %	2,43	2,31	2,48	2,49	2,25	2,50	2,22	1,85	2,32	2,45

4.3 Залежність часу V3 від K

Час V3 за P = 8 залежно від K наведено в табл. 5 і на рис. 2. У таблиці також наведено частку оновлень найбільш навантаженої секції і величину Σf^2, яка дорівнює ймовірності того, що два випадкові оновлення звертаються до однієї блокувальної змінної (f позначає частку оновлень секції).

Таблиця 5 – Час V3 і навантаження блокувальних змінних залежно від K (P = 8)

K	**1**	**4**	**16**	**64**	**256**	**1024**	**4096**
Медіана, с	10,909	6,681	3,324	2,297	1,718	1,540	1,489
Мін. – макс., с	10,79 – 11,18	6,49 – 7,54	3,31 – 3,67	2,26 – 2,33	1,70 – 1,75	1,52 – 1,56	1,47 – 1,56
Частка найбільш навантаженої секції, %	100	29,94	12,63	8,79	7,77	7,64	7,61
Σf^2	1	0,2570	0,0722	0,0245	0,0128	0,0102	0,0096

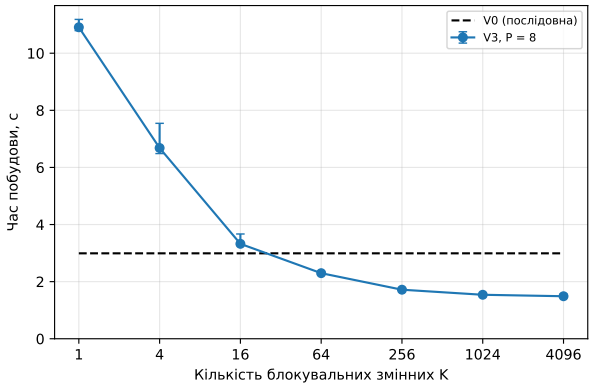


Рисунок 2 – Час виконання V3 залежно від K (P = 8)

Для перевірки пояснень виконано два контрольні вимірювання (скрипт `scripts/supplementary.sh`). На копії корпусу без 20 найчастотніших слів V3 за P = 4, 6 і 8 працює 0,979, 0,956 і 0,809 с відповідно (час V0 на цьому корпусі становить 2,642 с, S(8) = 3,27). Без заповнювача в секціях V3 за P = 8 працює повільніше на 26 % за K = 16 (4,017 с проти 3,193 с), на 19 % за K = 256 і на 12 % за K = 1024.

4.4 Злиття у V4

Час злиття обома способами і частку злиття в загальному часі V4 зі злиттям деревом наведено в табл. 6.

Таблиця 6 – Злиття у V4

P	**2**	**4**	**6**	**8**	**10**
Послідовне злиття, с	0,165	0,296	0,382	0,451	0,503
Злиття деревом, с	0,167	0,285	0,345	0,368	0,407
Частка злиття (дерево)	9 %	24 %	36 %	44 %	48 %
Прискорення побудови локальних індексів	1,85	3,32	5,04	6,48	6,97

За оцінкою обсягу роботи злиття для P = 8, отриманою з індексу V0, на критичному шляху послідовна схема копіює 3 458 648 пар «документ – кількість» за 7 кроків, а схема дерева копіює 3 331 683 пари за 3 кроки (0,46, 0,94 і 1,93 млн на кроках). Максимальний обсяг пам’яті, використаної процесом, становить 624 МБ для V0, 626 МБ для V3, 912 МБ для V4 з послідовним злиттям і 992 МБ для V4 зі злиттям деревом. Вісім локальних індексів разом містять 853 436 записів слів, а результуючий індекс містить 495 761.

4.5 Хибне розділення

Час виконання для 10 млн інкрементів на потік, як передбачено завданням, наведено на рис. 3, а. За такої тривалості (3–9 мс) на результат помітно впливає створення горутин, тому висновки зроблено за вимірюванням для 200 млн інкрементів (табл. 7, рис. 3, б).

Таблиця 7 – Медіана часу, мс, для 200 млн інкрементів на потік

Відстань, байт	**8 (A)**	**32**	**40**	**64 (B)**	**96**	**128**	**256**
P = 1	64,5	62,1	64,5	65,9	62,2	62,6	62,2
P = 2	257,9	257,7	258,3	73,4	72,7	72,0	67,8

Відстань, байт	8 (A)	32	40	64 (B)	96	128	256
P = 4	283,5	271,0	271,6	87,6	85,5	83,5	76,8
P = 8	337,3	314,0	311,8	283,3	121,3	87,5	87,3

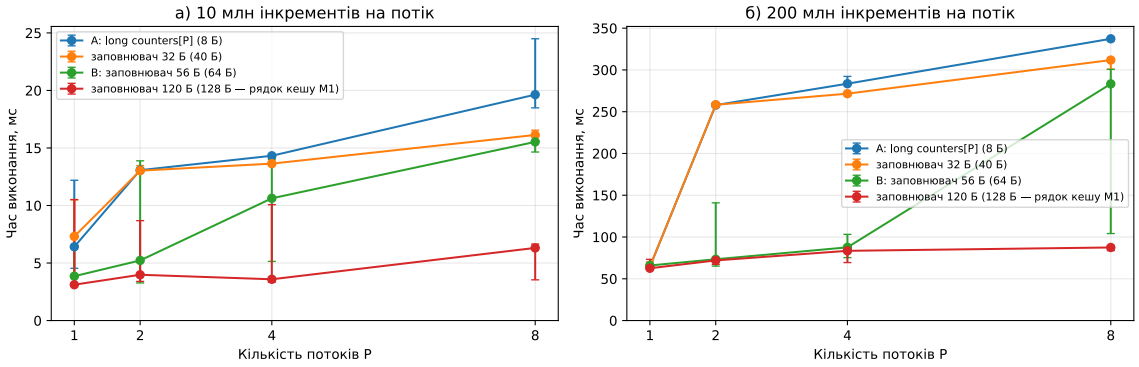


Рисунок 3 – Час виконання реалізацій A, B та інших відстаней між лічильниками для 10 млн (а) і 200 млн (б) інкрементів на потік

4.6 Послідовна частка

Значення метрики Карпа – Флатта e = (1/S − 1/P) / (1 − 1/P) наведено в табл. 8 і на рис. 4.

Таблиця 8 – Метрика Карпа – Флатта

P	V2	V3	V4, послідовне	V4, дерево	cas	syncmap
2	3,21	0,267	0,228	0,204	0,859	2,82
4	3,11	0,187	0,215	0,201	0,620	1,51
6	3,49	0,425	0,199	0,180	0,601	1,27
8	3,87	0,446	0,209	0,177	0,585	1,28
10	4,18	0,492	0,239	0,202	0,598	1,20

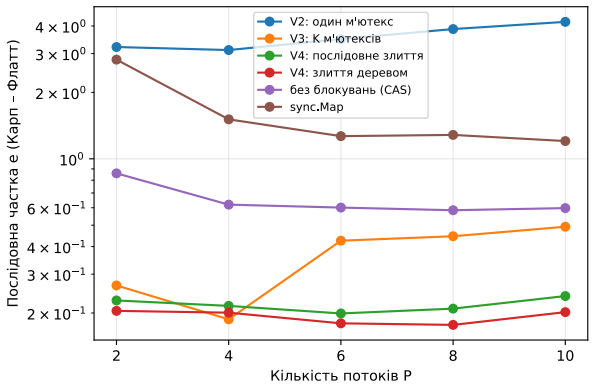


Рисунок 4 – Метрика Карпа – Флатта (логарифмічна шкала)

4.7 Профілювання

Розподіл процесорного часу за даними профілювальника (`-cpuprofile`, `go tool pprof`) наведено в табл. 9.

Таблиця 9 – Розподіл процесорного часу за даними профілювання

Версія	**Основні ділянки коду**
V0	`madvise` (отримання пам’яті від ОС) 24 %, вставлення в хеш-таблицю 20 %, нормалізація 16 %
V2, P = 8	простій ядер у планувальнику (`usleep`) 49 %, переведення горутин у стан очікування та їх пробудження (`pthread_cond_wait`, `pthread_cond_signal`) 35 %, повільний шлях виконання `Lock` 4 %, робота з хеш-таблицею менше 2 %
V3, P = 8	`usleep` 67 %, `pthread_cond_wait` 9 %, вставлення в хеш-таблицю 4 %
V4 (дерево), P = 8	`madvise` 50 %, вставлення в хеш-таблицю 8 %, нормалізація 6 %
`cas`, P = 8	`madvise` 33 %, пошук і вставлення в таблиці 13 %, нормалізація 9 %
`syncmap`, P = 8	`madvise` 36 %, обхід дерева `HashTrieMap` 10 %, обчислення хешів ключів інтерфейсного типу 9 %

4.8 Оптимізація нормалізації

На 20 книгах корпусу швидкість нормалізації становить 125 МБ/с для функції `Words` і 332 МБ/с для функції `FastWords`, тобто `FastWords` у 2,65 раза швидша. Час V0 становить 4,399 с з `Words` і 2,992 с з `FastWords` (у 1,47 раза менше); час V4 зі злиттям деревом за P = 8 становить відповідно 1,034 і 0,836 с (у 1,24 раза менше).

5. АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Версія V0

Близько чверті часу V0 займає системний виклик `madvise`, через який ОС надає програмі пам’ять у міру зростання купи (обсяг індексу близько 400 МБ). Ще п’ята частина часу припадає на вставлення в хеш-таблицю, а решту займають нормалізація та пошук.

5.2 Версія V1

Гонка даних виникає в рядку `shared.go:53` (`nd.count++`), який компілюється в три інструкції ARM64: читання (`MOVD 32(R1), R2`), збільшення (`ADD $1, R2, R2`) і запис (`MOVD R2, 32(R1)`). Якщо другий потік читає лічильник між читанням і записом першого, обидва потоки записують однакове значення, і одне оновлення втрачається. Лічильник кількості входжень у документ змінює лише один потік, а загальну кількість входжень слова змінюють усі потоки. Тому в контрольному запуску 94,4 % із 733 927 втрачених оновлень припали на 20 найчастотніших слів (з них 433 241 на слово `the`), а для 1 746 слів загальна кількість перестала дорівнювати сумі кількостей за документами. Друге місце гонки пов’язане з додаванням вузла в комірку (`shared.go:57`): потік читає голову ланцюжка, створює вузол і записує нову голову. Через велику кількість комірок ця гонка виникає рідко: у тому самому запуску втрачено один запис кількості входжень у документ. Обидва рядки виявляє детектор гонок Go. Частка втрат зростає з P, оскільки зростає кількість одночасних звертань до лічильників частих слів.

5.3 Версія V2

Уся робота з індексом виконується під захистом блокувальної змінної, а паралельно виконується лише нормалізація, на яку припадає менше п’ятої частини часу, тому за законом Амдала

прискорення не могло б перевищити приблизно 1,2. Фактично за $P \geq 2$ V2 працює повільніше за V0. За конкуренції горутина, яка не захопила `sync.Mutex` після короткого активного очікування, переводиться в стан очікування і пробуджується після його звільнення, а вільні ядра витрачають час у планувальнику. Корисна робота становить менше 5 % процесорного часу; крім того, рядки кешу з блокувальною змінною і зміненими записами індексу щоразу передаються між ядрами. Значення метрики Карпа – Флатта більше за 1 і зростає з P: накладні витрати перевищують корисну роботу.

5.4 Версія V3

Зі зростанням K зменшується ймовірність звернення двох оновлень до однієї блокувальної змінної (Σf^2 спадає з 1 до 0,0102), тому час зменшується. Після $K \approx 1024$ час практично не змінюється: перехід до K = 4096 дає лише 3 % (розкиди перетинаються), а Σf^2 зменшується на 6 % проти 20 % між K = 256 і K = 1024. Межу визначає розподіл частот слів: усі оновлення одного слова захищає одна блокувальна змінна, тому найбільш навантажена секція за будь-якого K отримує не менше 7,61 % оновлень (частка слова `the`), а вже за K = 256 її частка становить 7,77 %. Подальше збільшення K розподіляє лише рідкісні слова, оновлення яких майже не конфліктують.

Прискорення V3 зростає до P = 4 (2,56) і зменшується за $P \geq 6$ (1,92–1,94), а значення e зростає з 0,19 до 0,45: прискорення обмежують накладні витрати на синхронізацію, а не послідовна частина програми. За даними профілювання 67 % процесорного часу припадає на простій ядер у планувальнику, тобто горутини регулярно очікують звільнення блокувальних змінних найбільш навантажених секцій. Контрольне вимірювання підтверджує цю причину: на корпусі без 20 найчастотніших слів V3 масштабується до восьми потоків (S = 3,27) і не має спаду між P = 4 і P = 6. Спад на повному корпусі, імовірно, пояснюється будовою M1 Max: до чотирьох потоків можуть виконуватися в одному P-кластері зі спільним кешем L2, а за P > 4 рядки кешу з блокувальними змінними і записами частих слів передаються між кластерами через системний кеш, що значно повільніше. Без заповнювача вісім блокувальних змінних розміщуються в одному рядку кешу, і через хибне розділення V3 працює повільніше на 12–26 %.

5.5 Версія V4

Побудова локальних індексів масштабується майже лінійно (прискорення 6,48 за P = 8, ефективність 0,81), оскільки потоки не звертаються до спільних даних. Прискорення обмежують злиття (44 % часу за P = 8) і отримання пам’яті для P локальних індексів: за даними профілювання на `madvise` припадає половина процесорного часу. Злиття деревом швидше за послідовне лише на 18 %, хоча має 3 кроки замість 7. На k-му рівні дерева зливаються індекси, кожен з яких уже містить дані 2^(k−1) локальних індексів, тому вартість рівнів зростає у співвідношенні 1 : 2 : 4, і для даних, які не повторюються в різних потоках, критичний шлях дерева становить 7 одиниць, як і в послідовної схеми. Модель це підтверджує (3,33 млн проти 3,46 млн скопійованих пар). Перевага дерева зумовлена тим, що часті слова, спільні для всіх потоків, об’єднуються паралельно на нижніх рівнях, а менші хеш-таблиці нижніх рівнів краще вміщуються в кеш. За P = 10 додаються два повільніші E-ядра, і час не зменшується.

5.6 Версія без блокувань і версія на основі sync.Map

У версії `cas` немає блокувальних змінних, і ядра не простоюють у планувальнику, проте за P = 8 вона працює повільніше (1,90 с) за V3 (1,54 с) і V4 (0,84 с). Уже за P = 1 вона на 37 % повільніша за V0 через власну хеш-таблицю, атомарні операції для кожного токена, окремий вузол для кожної пари «слово, документ» і перетворення таблиці на індекс наприкінці. Масштабування обмежують ті самі часті слова: рядок кешу з лічильником слова `the`, який атомарно збільшують усі ядра, постійно передається між ними (істинне, а не хибне розділення даних), тому e ≈ 0,6 за будь-якого P. Версія `syncmap` найповільніша з коректних (8,9 с за P = 1 і 3,5 с за P = 10): ключі та значення `sync.Map` мають тип `any`, тому для кожного входження слова створюється рядок і виділяється пам'ять під ключ інтерфейсного типу, а обчислення хешу такого ключа й обхід дерева повільніші, ніж у типізованій `map`. Структура `sync.Map` оптимізована для ключів, які записуються один раз і багато разів читаються, тоді як побудова індексу складається з постійних записів у ті самі ключі.

5.7 Хибне розділення

Один потік виконує 200 млн інкрементів приблизно за 63 мс за будь-якого розміщення лічильників: рядок кешу з лічильником перебуває в кеші L1 у стані *Modified*, і інкремент виконується приблизно за один такт. Протокол когерентності кешів працює з цілими рядками: щоб записати дані, ядро має отримати виключний доступ до рядка, а копії рядка в кешах інших ядер стають недійсними. Якщо лічильники різних потоків лежать в одному рядку, кожен запис змушує інше ядро повторно завантажувати рядок, тому реалізація A працює повільніше, ніж за P = 1, у 4 рази за P = 2 і у 5,2 раза за P = 8, хоча потоки не мають спільних даних. Хибне розділення зникає лише за відстані 128 байт між лічильниками (за P = 8 час для 128 і 256 байт однаковий), що відповідає розміру рядка кешу M1 Max. Реалізація B розрахована на рядок кешу 64 байти, типовий для процесорів x86, і на M1 недостатня: за P = 8 вона лише на 16 % швидша за A, бо сусідні лічильники лежать в одному 128-байтному рядку. За P = 2–4 відстань 64 байти майже не уповільнює роботу; імовірно, це пов'язано з тим, що потоки виконуються в одному кластері зі спільним кешем L2. Отже, розмір заповнювача має дорівнювати розміру рядка кешу за вирахуванням розміру лічильника: 120 байт для M1 і 56 байт для процесорів із 64-байтним рядком.

5.8 Закон Амдала й оптимізація нормалізації

Для V4 зі злиттям деревом метрика Карпа – Флатта майже не залежить від P (0,18–0,20), тобто відповідає справжній послідовній частині програми: злиттю і послідовним частинам отримання пам'яті. За законом Амдала граничне прискорення становить $S(\infty) = 1/e \approx 5{,}6$. Для V2 і V3 значення e зростає з P, отже, прискорення обмежують накладні витрати на синхронізацію: прогноз для V3 за значенням e за P = 2 ($S(8) = 2{,}79$) перевищує фактичне значення 1,94. Для V4 фактичне прискорення $S(8) = 3{,}58$ більше за прогноз 3,29, оскільки частина накладних витрат за P = 2 не залежить від кількості потоків. Відхилення від ідеального прискорення зумовлюють злиття, синхронізація на частих словах, отримання пам'яті від ОС, наявність двох кластерів P-ядер і використання двох повільніших E-ядер за P = 10.

Функція `FastWords` обробляє ASCII-текст по 8 байтів за ітерацію методом SWAR (SIMD within a register): вісім символів класифікуються кількома арифметичними операціями над одним 64-

бітним словом, а слова з малих літер ASCII повертаються без копіювання. Оптимізація прискорює нормалізацію у 2,65 раза, версію V0 у 1,47 раза, а V4 за P = 8 лише у 1,24 раза: нормалізація добре розпаралелюється, тому її прискорення найбільше впливає на послідовну версію.

6. ВИСНОВКИ

– Продуктивність визначає не кількість потоків, а частота звертань потоків до спільних даних. Найшвидшою є версія V4 зі злиттям деревом (S = 3,58 за P = 8), у якій потоки працюють без спільних даних, а синхронізацію замінює злиття.
– Одна блокувальна змінна для всього індексу (V2) робить оновлення послідовними і додає витрати на переведення потоків у стан очікування та їх пробудження, тому V2 повільніша за послідовну версію у 2,1–3,9 раза.
– Секціоноване блокування (V3) ефективне, доки збільшення K розподіляє часті слова між різними секціями. За природного розподілу частот слів кілька найчастотніших слів обмежують прискорення за будь-якого K.
– Відмова від блокувальних змінних (`cas`) не усуває конкуренції за лічильники частих слів, а універсальна конкурентна структура даних (`sync.Map`) повільніша за спеціалізоване рішення.
– Розміщення даних у пам’яті суттєво впливає на продуктивність: хибне розділення уповільнює роботу в 4–5 разів, а рядок кешу M1 Max (128 байт) удвічі більший, ніж передбачає реалізація B.

7. ДЖЕРЕЛА ТА ІНСТРУМЕНТИ

– Project Gutenberg. Robot access policy: https://www.gutenberg.org/policy/robot_access.html; дзеркало PGLAF: https://gutenberg.pglaf.org.
– The Go Programming Language Specification; The Go Memory Model: https://go.dev/ref/mem; документація стандартної бібліотеки Go (`sync`, `sync/atomic`, `hash/maphash`, `bytes`, `unicode`, `runtime/pprof`, `testing`).
– Інструменти: Go 1.26.2 (`go test -race`, фазинг-тестування, `go tool pprof`, `go tool objdump`), Python 3 і matplotlib (графіки), pandoc і Typst (звіт). Запозиченого коду немає.

ДОДАТОК А. ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАПИТАННЯ

1) **Рядок коду V1, у якому виникає гонка даних.** Рядок `shared.go:53` (`nd.count++`): читання, збільшення і запис лічильника. Оновлення втрачається, якщо другий потік читає лічильник між читанням і записом першого. Аналогічна гонка між читанням і записом голови ланцюжка виникає в рядку `shared.go:57` (підрозд. 5.2).
2) **Чому V2 на восьми потоках працює повільніше за V0.** Оновлення залишаються послідовними, але кожне з них дорожче: горутини переводяться в стан очікування і пробуджуються, ядра простоюють у планувальнику, рядки кешу передаються між ядрами. Корисна робота становить менше 5 % процесорного часу (10,5 с проти 3,0 с; підрозд. 5.3).

3) **Як зміниться час V3 за збільшення K від 1024 до 4096.** Час практично не зміниться (1,540 і 1,489 с, у межах розкиду): часті слова вже розподілені між різними секціями, найбільш навантажена секція отримує 7,6 % оновлень за будь-якого K, а Σf^2 зменшується лише з 0,0102 до 0,0096.
4) **Яку частку оновлень спричиняють 20 найчастотніших слів.** 26,75 % (8 634 928 із 32 284 949), зокрема 7,61 % припадає на слово `the`. Ці оновлення конкурують за ті самі блокувальні змінні за будь-якого K, тому час на рис. 2 зменшується, поки збільшення K розподіляє часті слова між секціями, і перестає зменшуватися після $K \approx 1024$.
5) **Які накладні витрати виникають у V4 замість витрат на синхронізацію.** Злиття локальних індексів (44–49 % часу за $P = 8$), яке частково виконується послідовно; додаткова пам'ять для частих слів у кожному локальному індексі (912–992 МБ проти 624 МБ у V0) та її отримання від ОС; очікування завершення найповільнішого потоку перед злиттям.
6) **Різниця часу двох способів злиття.** Злиття деревом швидше лише на 18 % (0,368 с проти 0,451 с за $P = 8$), хоча має 3 кроки замість 7: вартість кожного рівня дерева вдвічі більша за вартість попереднього, тому для даних, які не повторюються в різних потоках, критичний шлях обох схем становить $P - 1$ одиниць (підрозд. 5.5).
7) **Як зміниться результат за заповнювача 32 байти.** Елемент займатиме 40 байт, сусідні лічильники лежатимуть в одному рядку кешу, і хибне розділення майже не зменшиться: 258 мс за $P = 2$, як у реалізації A, і 312 мс за $P = 8$ проти 88 мс за відстані 128 байт. На M1 заповнювач має бути не меншим за 120 байт.
8) **Оцінка ефективності на восьми потоках.** Найвищу ефективність (0,45) має V4 зі злиттям деревом: побудова локальних індексів має ефективність 0,81, а злиття займає 44 % часу. За $e \approx 0{,}18$ закон Амдала дає для $P = 8$ граничне прискорення близько 3,6, що відповідає виміряному значенню. Ефективність інших версій становить від 0,04 (V2) до 0,24 (V3) через синхронізацію на частих словах.